

# CSW2 ASSIGNMENT 4

## (CHAPTER 15 – GARBAGE COLLECTION)

### SOLUTIONS

---

1. Write a Java program to demonstrate garbage collection using an **UnreachableObject** class. It should include a constructor to initialize an object with a given name, a **show()** method creating an instance and invoking **display()**, and the **display()** method creating another instance. The **main()** method should call **show()** and explicitly invoke the garbage collector. The program must override the **finalize()** method to print the object's name upon successful garbage collection, illustrating how unreachable objects are collected.

```
package q1;
public class UnreachableObject {
    private String name;
    public UnreachableObject(String name) {
        this.name = name;
        System.out.println(name + " created.");
    }
    public void show() {
        UnreachableObject obj = new UnreachableObject("InnerObject1");
        obj.display();
        obj = null;
    }
    public void display() {
        UnreachableObject obj = new UnreachableObject("InnerObject2");
        obj = null;
    }
    protected void finalize() throws Throwable {
        System.out.println(name + " is garbage collected.");
        super.finalize();
    }
    public static void main(String[] args) {
        UnreachableObject obj = new UnreachableObject("OuterObject");
        obj.show();
        obj = null;
        System.gc();
    }
}
```

#### OUTPUT:

```
OuterObject created.
InnerObject1 created.
InnerObject2 created.
OuterObject is garbage collected.
InnerObject2 is garbage collected.
InnerObject1 is garbage collected.
```

2. Develop a Java program to demonstrate reference reassignment and garbage collection using the **ReassigningReference** class. The class should have a constructor to initialize an object with a given name. In the **main()** method, create two instances of **ReassigningReference**, reassign one reference to another, and then explicitly invoke the garbage collector. Override the **finalize()** method to print the object's name upon successful garbage collection.

```
package q2;
public class ReassigningReference {
    private String name;
    public ReassigningReference(String name) {
        this.name = name;
        System.out.println(name + " created.");
    }
    protected void finalize() throws Throwable {
        System.out.println(name + " is garbage collected.");
        super.finalize();
    }
    public static void main(String[] args) {
        ReassigningReference obj1 = new ReassigningReference("Object1");
        ReassigningReference obj2 = new ReassigningReference("Object2");
        obj1 = obj2;
        System.gc();
    }
}
```

#### **OUTPUT:**

```
Object1 created.
Object2 created.
Object1 is garbage collected.
```

3. Write a Java program to demonstrate nullification of references and garbage collection using the **NullifiedReference** class. The class should have a constructor to initialize an object with a given name. In the **main()** method, create an instance of **NullifiedReference**, assign it, then nullify the reference, and explicitly invoke the garbage collector. Override the **finalize()** method to print the object's name upon successful garbage collection.

```
package q3;
public class NullifiedReference {
    private String name;
    public NullifiedReference(String name) {
        this.name = name;
        System.out.println(name + " created.");
    }
    protected void finalize() throws Throwable {
        System.out.println(name + " is garbage collected.");
        super.finalize();
    }
    public static void main(String[] args) {
        NullifiedReference obj = new NullifiedReference("Object1");
        obj = null;
        System.gc();
    }
}
```

#### **OUTPUT:**

```
Object1 created.
Object1 is garbage collected.
```

4. Create a Java program to demonstrate anonymous objects and garbage collection using the **AnonymousObject** class. The class should have a constructor to initialize an object with a name. In the **main()** method, create an anonymous object and explicitly invoke the garbage collector. Override the **finalize()** method to print the object's name upon successful garbage collection.

```
package q4;
public class AnonymousObject {
    private String name;
    public AnonymousObject(String name) {
        this.name = name;
        System.out.println(name + " created.");
    }
    protected void finalize() throws Throwable {
        System.out.println(name + " is garbage collected.");
        super.finalize();
    }
    public static void main(String[] args) {
        new AnonymousObject("AnonymousObject");
        System.gc();
    }
}
```

#### **OUTPUT:**

```
AnonymousObject created.
AnonymousObject is garbage collected.
```

5. Develop a Java class with private integer and double data members, along with methods for initialization, setting, and updating these members. Create two objects of this class and invoke the necessary methods to modify the data. Use the **Runtime** class to calculate the total allocated memory and memory occupied by the objects. Apply any technique to make the objects unreachable, making them eligible for garbage collection. Finally, recheck the utilized and total memory using the Runtime class.

```
package q5;
public class MemoryUsage {
    private int intVal; private double dblVal;
    public MemoryUsage(int intVal, double dblVal) {
        this.intVal = intVal; this.dblVal = dblVal;
    }
    public void setintVal(int intVal) {
        this.intVal = intVal;
    }
    public void setdblVal(double dblVal) {
        this.dblVal = dblVal;
    }
    public void displayValues() {
        System.out.println("[Integer Value: " + intVal + ", Double Value: " +
dblVal + "]");
    }
    private static void displayMemoryStatus(String message) {
        Runtime runtime = Runtime.getRuntime();
        long totalMemory = runtime.totalMemory();
        long freeMemory = runtime.freeMemory();
        long usedMemory = totalMemory - freeMemory;
        System.out.println(message);
        System.out.println("Total Memory: " + totalMemory + " | Free Memory: " +
freeMemory + " | Used Memory: " + usedMemory);
    }
    public static void main(String[] args) {
        displayMemoryStatus("Initial Memory Status:");
        MemoryUsage obj1 = new MemoryUsage(10, 20.5);
        MemoryUsage obj2 = new MemoryUsage(30, 40.8);
        obj1.setintVal(15); obj1.setdblVal(25.5);
        obj2.setintVal(35); obj2.setdblVal(45.5);
        System.out.print("\nObject 1: ");
        obj1.displayValues();
        System.out.print("Object 2: ");
        obj2.displayValues();
        displayMemoryStatus("\nMemory Status After Object Creation:");
        obj1 = null; obj2 = null;
        System.gc();
        displayMemoryStatus("\nMemory Status After Garbage Collection:");
    }
}
```

### OUTPUT:

```
Initial Memory Status:
Total Memory: 132120576 | Free Memory: 130587576 | Used Memory: 1533000

Object 1: [Integer Value: 15, Double Value: 25.5]
Object 2: [Integer Value: 35, Double Value: 45.5]

Memory Status After Object Creation:
Total Memory: 132120576 | Free Memory: 130587576 | Used Memory: 1533000

Memory Status After Garbage Collection:
Total Memory: 8388608 | Free Memory: 7795616 | Used Memory: 592992
```

6. Write a memory-intensive Java program that creates a large number of objects and test it using the G1 garbage collector. Print timestamps along with the total heap size and free memory. Use the following commands to retrieve heap memory details:

- **Total heap memory:** `Runtime.getRuntime().totalMemory();`
- **Free heap memory:** `Runtime.getRuntime().freeMemory();`

```
package q6;
import java.util.ArrayList;
import java.util.Date;
import java.util.List;
public class MemoryIntensiveProgram {
    public static void main(String[] args) {
        List<Object> objectList = new ArrayList<>();
        printMemoryDetails("Initial");
        try {
            for (int i = 0; i < 1000; i++) {
                objectList.add(new byte[1024]);
                if (i % 500 == 0) {
                    printMemoryDetails("After creating " + i + " objects");
                }
            }
        } catch (OutOfMemoryError e) {
            System.out.println("Out of memory! The program generated too many
objects.");
        }
        objectList.clear(); System.gc();
        printMemoryDetails("Final");
    }
    private static void printMemoryDetails(String phase) {
        long totalMemory = Runtime.getRuntime().totalMemory();
        long freeMemory = Runtime.getRuntime().freeMemory();
        long usedMemory = totalMemory - freeMemory;
        System.out.println "[" + new Date() + "] " + phase);
        System.out.println("Total Memory: " + totalMemory / 1024 + " KB");
        System.out.println("Free Memory: " + freeMemory / 1024 + " KB");
        System.out.println("Used Memory: " + usedMemory / 1024 + " KB");
    }
}
```

### OUTPUT:

```
[Tue Mar 18 21:55:25 IST 2025] Initial
Total Memory: 129024 KB
Free Memory: 127526 KB
Used Memory: 1497 KB
-----
[Tue Mar 18 21:55:25 IST 2025] After creating 0 objects
Total Memory: 129024 KB
Free Memory: 126502 KB
Used Memory: 2521 KB
-----
[Tue Mar 18 21:55:25 IST 2025] After creating 500 objects
Total Memory: 129024 KB
Free Memory: 126030 KB
Used Memory: 2993 KB
-----
[Tue Mar 18 21:55:25 IST 2025] Final
Total Memory: 10240 KB
Free Memory: 8802 KB
Used Memory: 1437 KB
-----
```

7. Create a Java program for university student enrollment using a **Student** class to manage course details and student information. Implement efficient garbage collection for memory management and use the **Runtime** class to monitor memory usage. Override the **finalize()** method to print a message upon successful garbage collection.

```
package q7;
public class Student {
    private String studentName; private String courseName;
    public Student(String studentName, String courseName) {
        this.studentName = studentName; this.courseName = courseName;
    }
    public void displayStudentDetails() {
        System.out.println("Student Name: " + studentName + ", Course Name: " +
courseName);
    }
    protected void finalize() throws Throwable {
        System.out.println("Garbage collector called for student: " +
studentName);
        super.finalize();
    }
    private static void displayMemoryUsage(String message, Runtime runtime) {
        long totalMemory = runtime.totalMemory();
        long freeMemory = runtime.freeMemory();
        long usedMemory = totalMemory - freeMemory;
        System.out.println(message);
        System.out.println("Total Memory: " + totalMemory + " | Free Memory: " +
freeMemory + " | Used Memory: " + usedMemory);
    }
    public static void main(String[] args) {
        Runtime runtime = Runtime.getRuntime();
        displayMemoryUsage("Initial Memory Status:", runtime);
        Student student1 = new Student("Anindya", "Physics");
        Student student2 = new Student("Sayan", "Mathematics");
        System.out.println("\nStudent 1 Details:");
        student1.displayStudentDetails();
        System.out.println("Student 2 Details:");
        student2.displayStudentDetails();
        displayMemoryUsage("\nMemory Status After Object Creation:", runtime);
        student1 = null; student2 = null;
        System.gc();
        displayMemoryUsage("\nMemory Status After Garbage Collection:",
runtime);
    }
}
```

### OUTPUT:

```
Initial Memory Status:
Total Memory: 132120576 | Free Memory: 130587576 | Used Memory: 1533000

Student 1 Details:
Student Name: Anindya, Course Name: Physics
Student 2 Details:
Student Name: Sayan, Course Name: Mathematics

Memory Status After Object Creation:
Total Memory: 132120576 | Free Memory: 130587576 | Used Memory: 1533000

Memory Status After Garbage Collection:
Total Memory: 8388608 | Free Memory: 7836280 | Used Memory: 552328
Garbage collector called for student: Sayan
Garbage collector called for student: Anindya
```